CMPT 981
Quantum Circuits & Compilation

Final project Report Sara Babaee

1 Introduction

As quantum computing continues to develop, efficient simulation of quantum circuits remains an essential tool for both theoretical and practical progress. There are two primary motivations for simulating quantum circuit: first, it plays a crucial role in circuit design, testing, and debugging; second, it is widely used in quantum physics and chemistry, where researchers model complex quantum systems and study their behaviors using quantum circuits. [1]

Despite these important applications, general-purpose simulation of quantum circuits is computationally expensive due to the exponential growth of the state space with the number of qubits. While there is no known efficient classical method for simulating arbitrary quantum circuits, the Gottesman-Knill theorem shows that circuits composed entirely of Clifford operations (Hadamard, Phase, and CNOT gates) and computational basis measurements—known as stabilizer circuits—can be simulated in polynomial time on a classical computer. This stabilizer formalism is central not only to simulation but also to emerging applications in quantum error correction and quantum program analysis.

One of the most practical and well-known approaches is the Tableau algorithm proposed by Scott Aaronson and Daniel Gottesman [1], which enables fast classical simulation of stabilizer circuits involving thousands of qubits.

In this project, I implemented the Tableau algorithm from [1] in Haskell and explored its use for efficient simulation of stabilizer circuits. The implementation was tested on various Clifford circuits and compared to the existing C implementation provided by [1].

2 Implementation Process

The implementation is written in Haskell and organized into four modules: `Quantum`, `PauliOperator`, `StabilizerSimulation`, and `Main`. Each module handles a specific part of the simulation, from defining core quantum data types to performing stabilizer updates and running the main simulation loop.

Quantum Module

The `Quantum` module defines the core algebraic data types for representing quantum operations:

`Gate`: Supports `CX` (controlled-X), `H` (Hadamard), and `S` (Phase gate).

`Measure`: Represents a measurement operation with a single value `M`.

`QuantumInstruction`: Represents an instruction in the circuit, which can be a `QuantumOp` (containing a `Gate`), a `ClassicOp` (containing a `Measure`), or `Skip`.

`Circuit`: A complete quantum circuit, with three fields:

- `qubitNum'`: number of qubits,
- `instructionNum`: number of instructions (including skips),
- `instructionSet`: list of instructions to be performed.

`MeasurementResult`: Indicates the outcome of a measurement, including:

- `qubitNum'`: index of the measured qubit,
- `result`: the result (0 or 1),
- `isRandom`: whether the result was random (i.e., the qubit was not in a computational basis state).

Main Module

This module contains the main IO function, which handles reading a `.chpqasm.txt` input file from the `./benchmarks/chpqasm/input/` directory. The file is parsed into a `Circuit`, which is then simulated while measuring the runtime using the `timeFunction`. Measurement results are written to an output file in `./benchmarks/chpqasm/output/`. The input file name (excluding postfix) is passed as a command-line argument.

Since random measurements require a source of randomness, a random generator (`StdGen`) is initialized from the OS and passed through the simulation chain. Due to Haskell's pure functional nature, random number generation has to be handled explicitly: each function that uses randomness takes and returns an updated generator.

StabilizerSimulation Module

This module defines the `Tableau` data type, which stores:

- the number of qubits (`n`),
- A list of `n` destabilizer Paulis,
- A list of `n` stabilizer Paulis.

Each `Pauli` operator is defined in the `PauliOperator` module.

To simulate a circuit, an initial `Tableau` is created with `n` qubits, and each instruction is applied in order. Measurements are stored in a first-in-last-out list of `MeasurementResult`.

PauliOperator Module

Defines the **`Pauli`** data type with:

- `xBits` and `zBits` as `BitVectors`,
- A `phaseBit`.

The use of `BitVector` instead of a list of `Bools` improves performance, especially for operations that involve frequent random access and bitwise operations.

Gate Operations

The `applyGate'` function applies a gate across the entire Tableau using the `applyGate` function (defined in `PauliOperator`) to each row. This function handles the gate transformation rules as described in the paper via pattern matching. Originally, a more modular version with auxiliary functions was used to reduce code repetition, but replacing that with a single function noticeably improved performance—saving about 0.5 seconds for a circuit with ~4000 gates.

It's expected that further improvements could be achieved by storing the Tableau in vertical `BitVectors` (i.e., column-wise), which could make gate operations faster but might complicate measurement operations.

Measurement Operations

To simulate a measurement on qubit `a`, the `measure` function in `StabilizerSimulation` checks if any stabilizer has a non-zero X at position `a`. If such a stabilizer exists, it's used as a pivot and the measurement is random; otherwise, a deterministic measurement is performed using `determinateMeasure` to determine whether the qubit is in $|0\rangle$ or $|1\rangle$.

The `rowsum(i, j)` operation from the paper is implemented as `groupOp` P_j P_i, which computes and returns the product $P_j \,.\, P_i$ as a new Pauli. This differs slightly from the paper's destructive update strategy ($P_i \leftarrow P_i \,.\, P_j$), so the order and usage were adjusted accordingly.

In random measurements, the `randomMeasure` function uses `measuringPauli` to simulate randomness. Since random generation in Haskell must be pure, the generator is threaded explicitly through each function call and updated manually.

All functions timed using `timeFunction` return results that are instances of `NFData`, and appropriate language extensions and instances were declared where needed.

3 Experimental Results

The implementation was first tested on small, hand-written circuits (including EPR pair, GHZ state, and teleportation). All tests produced the expected outcomes, indicating that the simulation logic was correctly implemented.

The simulator takes input in **CHPQASM**, a restricted Clifford-only fragment of OpenQASM [2]. A CHPQASM file must start with a fixed header, followed by a line declaring the number of qubits using `qreg <n>`, and then a sequence of instructions. The supported instructions are:

$$\langle \texttt{instr} \rangle \vDash \texttt{cx}\,\langle a \rangle \langle b \rangle \mid \texttt{h}\,\langle a \rangle \mid \texttt{s}\,\langle a \rangle \mid \texttt{measure}\,\langle a \rangle$$

$$\langle a \rangle \vDash 0 \mid 1 \mid \ldots \mid n-1$$

Each instruction should appear on its own line. Blank lines are interpreted as `Skip` instructions, except for the first two lines (header and qubit declaration), which must appear without any blank lines before or between them.

For performance comparison, my implementation was evaluated against the `chp.c` implementation from [1], available at [3]. To facilitate this comparison, the `randqc.c` program provided in [3] was modified to generate random Clifford circuits in both CHP and CHPQASM formats for consistency.

Results showed that my implementation, though correct and readable, is significantly slower. For instance, a 200-qubit, 4000-gate circuit (without any measurements) takes ~2 seconds to run in Haskell, while the C implementation completes gate simulation in effectively zero seconds.

As described in the implementation section, I took some steps to improve performance—such as replacing function chaining with a single gate-application function and using `BitVector` instead of lists for representing Paulis—but even with these changes, the performance gap remains large. That said, it still serves well for prototyping and educational purposes, and further optimizations (such as vertically storing the Tableau) might improve runtime for specific applications.

4 References

[1] Scott Aaronson and Daniel Gottesman. 2004. Improved simulation of stabilizer circuits. Physical Review A 70, 052328 (2004).

[2] Andrew W. Cross, Lev S. Bishop, John A. Smolin, Jay M. Gambetta, Open Quantum Assembly Language.

[3] At www.scottaaronson.com/chp

Number of qubits	**Number of gates**	**Average running time without measurement (s)**	**Average running time with measurement (s)**
100	1000	0.17	0.36
	2000	0.32	0.41
	3000	0.52	0.72
	4000	0.79	1.10
	5000	1.04	1.36
200	1000	0.38	0.82
	2000	0.88	1.74
	3000	1.52	1.97
	4000	2.13	2.70
	5000	3.00	2.90
300	1000	0.63	1.22
	2000	1.41	2.87
	3000	2.64	4.82
	4000	3.91	6.76
	5000	5.41	7.6

Table (1). Average running time of my implementation for Stabilizer circuits with 100 to 300 qubits and 1000 to 5000 gates, with and without measurement.

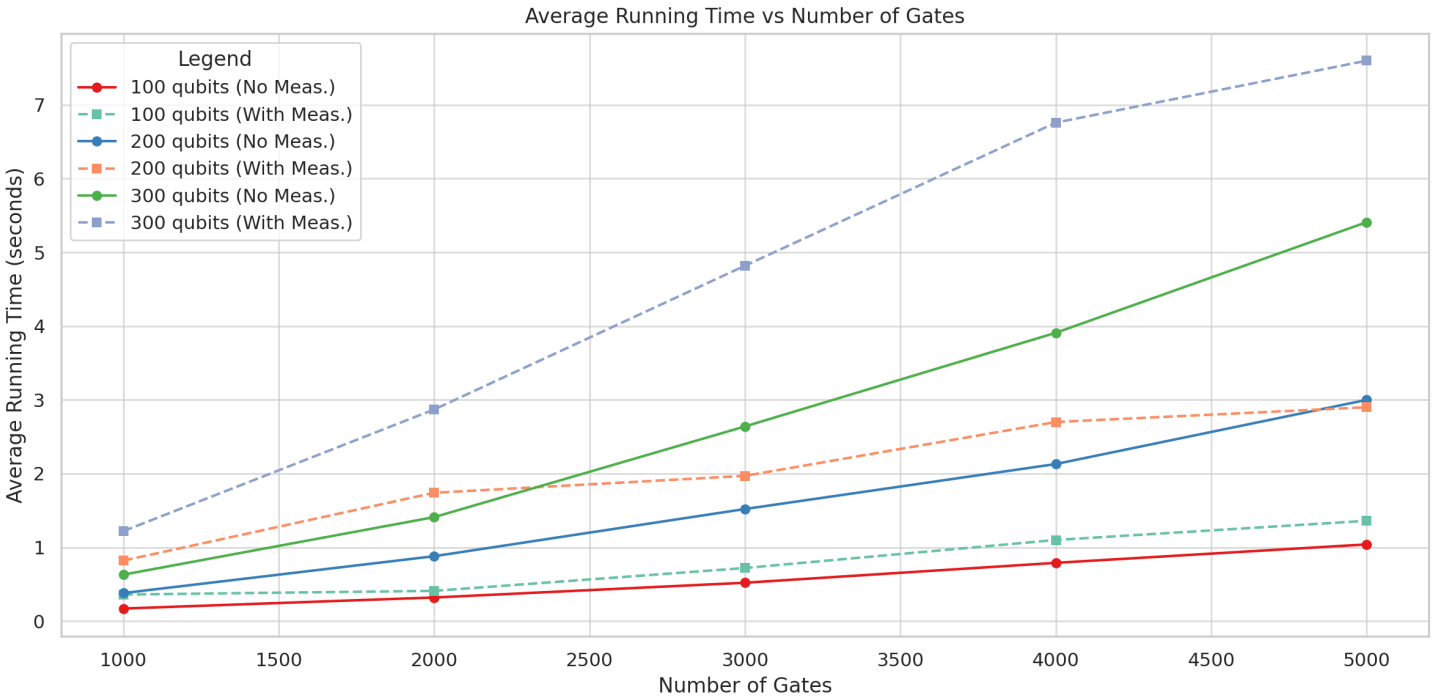

Fig (1). Graph of running time of experiment results.